

# **IoT-Enabled Aquaculture System with Feeding, Mortality Detection, Budgeting, Monitoring, and Alerts**

Project ID: 25-26J-295

Project Proposal Report

Sewwandi C.I

BSc (Hons) Degree in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2024

# **IoT-Enabled Aquaculture System with Feeding, Mortality Detection, Budgeting, Monitoring, and Alerts**

Project ID: 25-26J-295

## **Project Proposal Report**

BSc (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2024

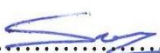
## DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

| Name         | Student ID | Signature                                                                           |
|--------------|------------|-------------------------------------------------------------------------------------|
| Sewwandi C.I | IT22609526 |  |

The above candidates are carrying out research for their undergraduate dissertation under my supervision.

Name of supervisor: Mrs. Uthpala Samarakoon

  
.....

Signature of the supervisor

29/08/2025  
.....

Date

## ABSTRACT

This undergraduate research proposal introduces an Internet of Things (IoT) system designed for the early detection of fish mortality and abnormal behaviour in aquaculture ponds, enhanced by environmental alerts. The proposed solution combines a Raspberry Pi B+ with an infrared (IR) camera and several water-quality sensors.

sensors (humidity, and rainfall) to keep an eye on the health of the fish and the state of the pond in real time. The system constantly watches the fish through a camera feed to look for unusual behaviour, like long periods of inactivity or strange swimming patterns that could mean the fish is sick or dead. At the same time, it keeps an eye on environmental factors, such as changes in water temperature and pH levels caused by events like rain, so it can send out warnings in a timely manner. When fish behave strangely or the weather is bad, alerts are sent to farmers' phones through a cloud service (Firebase), allowing them to act quickly. The research problem tackles the absence of cost-effective, proactive monitoring instruments in small-scale aquaculture capable of early detection of fish mortalities and rapid response to abrupt environmental shifts. The goals are to create the IoT hardware setup, write the algorithms for image processing and sensor data for finding anomalies, and set up a notification module that works with the cloud. The method includes making a prototype with a Raspberry Pi and the required sensors, writing software to collect and analyse data, and testing it in conditions that are like those of a pond. This proposal examines relevant literature regarding IoT in aquaculture and the analysis of fish behaviour, indicates the research gap in current solutions, and presents a timeline, work breakdown, and necessary budget. The goal of the outcome is to make fish farm management better by lowering the number of fish deaths that go unnoticed and lowering environmental risks through real-time alerts. This will lead to smarter and more effective aquaculture practices.

*Keywords: IOT, Fish Mortality Detection, Environmental Monitoring, Abnormal Behaviour, Raspberry Pi*

## Table of Contents

|                                                  |     |
|--------------------------------------------------|-----|
| DECLARATION .....                                | iii |
| ABSTRACT.....                                    | iv  |
| LIST OF FIGURES .....                            | vi  |
| LIST OF TABLES .....                             | vi  |
| LIST OF ABBREVIATIONS .....                      | vi  |
| 1 INTRODUCTION .....                             | 1   |
| 1.1 Background study .....                       | 1   |
| 1.2 Literature survey .....                      | 2   |
| 1.3 Research gap .....                           | 4   |
| 1.4 Research problem.....                        | 7   |
| 2 RESEARCH OBJECTIVE .....                       | 7   |
| 2.1 Main objective.....                          | 7   |
| 2.2 Specific objective .....                     | 7   |
| 3 METHODOLOGY .....                              | 8   |
| 3.1 System overview diagram .....                | 11  |
| 3.2 Component overview diagram .....             | 12  |
| 3.3 Gantt chart.....                             | 12  |
| 4 PROJECT REQUIREMENTS .....                     | 13  |
| 4.1 User requirements .....                      | 13  |
| 4.2 Functional requirements .....                | 13  |
| 4.3 Non - functional requirements .....          | 14  |
| 4.4 Test cases .....                             | 16  |
| 5 BUDGET AND JUSTIFICATION.....                  | 16  |
| 6 COMMERCIALIZATION AND MARKET FEASIBILITY ..... | 17  |
| 7 REFERENCES.....                                | 18  |
| 8 APPENDIX .....                                 | 19  |

## LIST OF FIGURES

|                                      |    |
|--------------------------------------|----|
| Figure 3.1: System overview .....    | 12 |
| Figure 3.2: component overview ..... | 12 |
| Figure 3.3: Gantt chart .....        | 12 |

## LIST OF TABLES

|                                           |    |
|-------------------------------------------|----|
| Table 1.1: Comparison chart .....         | 6  |
| Table 4.1: Test case .....                | 16 |
| Table 5.1: Budget and justification ..... | 16 |

## LIST OF ABBREVIATIONS

| Abbreviation | Description                       |
|--------------|-----------------------------------|
| ADC          | Analog-to-Digital Converter       |
| API          | Application Programming Interface |
| BMS          | Battery Management System         |
| CSI          | Camera Serial Interface           |
| KPI          | Key Performance Indicator         |
| LDR          | Light-Dependent Resistor          |
| RAS          | Recirculating Aquaculture System  |
| VPS          | Virtual Private Server            |
| FCM          | Firebase Cloud Messaging          |

*Table 4.1: Test case*

# 1 INTRODUCTION

## 1.1 Background study

Aquaculture has become one of the most rapidly growing parts of global food production. It helps with food security and economic growth in a big way. Small to medium-sized fish farmers, on the other hand, have a lot of problems when it comes to running their businesses, such as finding out about fish deaths early on and figuring out what environmental stressors are causing them. Traditional ways of keeping an eye on things, like manual visual inspection,[1] are time-consuming, unreliable, and often miss early signs of fish distress. Because of this delay in finding things, people die, diseases spread, and money is lost. Also, sudden changes in the environment, like rain and temperature changes, can have a big effect on water quality parameters like pH and dissolved oxygen, which can directly affect the health of fish.

Recent advancements IoT and computer vision [2] have created novel prospects for the automation of fish health monitoring. IoT devices let you constantly and remotely watch aquaculture systems by combining environmental sensors with cameras that can analyse behaviour. Vision-based monitoring systems can spot strange behaviours in fish, like floating, being lazy, or swimming in an unusual way, which often happens before death. When used with environmental monitoring that considers the current situation, these systems can send farmers accurate, event-driven alerts that let them act quickly.

Even with these improvements, small-scale cultivators still don't use these kinds of technologies frequently because they are too expensive, hard to operate, and there aren't any all-in-one solutions that combine fish behaviour analysis with environmental event detection. This proposal fixes these problems by creating an affordable, all-in-one Fish Mortality Detection and Environmental Alert System that is made just for

small-scale aquaculture operations. The system uses an edge computing platform based on a Raspberry Pi, which has an infrared camera and environmental sensors, to keep an eye on the health of fish and important environmental events in real time.

## **1.2 Literature survey**

### **IoT-based monitoring in aquaculture**

IoT architectures allow for continuous, remote monitoring of ponds by streaming core water parameters like pH and temperature [3]. This changes farm management from periodic, manual checks to proactive intervention. Recent research indicates that economical sensor networks with cloud backends enhance the early identification of deteriorating conditions and reduce response times to risk events in aquaculture settings [4] (Shete et al., 2024; Jais et al., 2024; Haemal et al., 2024). These studies demonstrate that instrumented ponds, enhanced by basic analytics and alert systems, can mitigate unnoticed stress and mortality while remaining economically viable for smallholders.

### **Computer vision for fish mortality and behaviour analysis**

Behavioural cues prolonged surface floating, stagnation, and atypical motion—are reliable precursors to mortality and can be detected using vision pipelines on farm video. In intensive systems, MortCam successfully generated real-time, automated mortality alerts directly from camera imagery, illustrating the viability of camera-first monitoring in production aquaculture (Ranjan et al., 2023). Recent research has adapted contemporary object detection and tracking techniques [5] (e.g., YOLO variants) to identify abnormal behaviours in real time, enhancing sensitivity to early stress while maintaining computational efficiency for deployment[6] (Xiao et al., 2024; Li et al., 2024).



### **Event-triggered environmental alerts (rainfall)**

Environmental shocks can change the chemistry of a pond in just a few minutes. For example, rainwater can dilute and acidify the water, which can lower the amount of dissolved oxygen and raise the pH to dangerous levels. Longitudinal pond studies show that nutrients and physicochemical variables change in measurable ways during rainy times. This shows that monitoring should be done in a way that takes the situation into account and that pH [7] and temperature should be checked more often as soon as rain is detected (Sipaúba-Tavares & Braga, 2007; Makori et al., 2017) [8]. Checks that are triggered by events cut down on unnecessary continuous sampling and focus on the times when fish are most likely to be stressed.

### **Edge computing for low-cost deployment**

Processing on an edge device (like a Raspberry Pi or ESP-class board) [4] lets you get data from sensors at the same time, take pictures at regular intervals, do light motion analysis, [10] and check thresholds with very little bandwidth and power. Research indicates that relocating inference to the edge, while transmitting only succinct alerts and summaries to the cloud, diminishes latency and data expenses, all the while maintaining prompt detection for single-pond farmers (Shete et al., 2024; Miller et al., 2023; Haemal et al., 2024). This design pattern directly supports systems that are cheap, easy for farmers to use, and can be set up without special infrastructure.

### **Cloud messaging and notification interfaces for farmers**

Once anomalies are confirmed, cloud messaging services send short mobile notifications that include context [9] (timestamp, current readings, and, optionally, an image), which makes it possible to triage from a distance. Small teams that use on-device detection and app-based notifications say they react faster and are more aware

of what's going on (Darmalim et al., 2021; "Real-time Water Quality Monitoring and Notification System for Aquaculture," 2021). Alert-centric interfaces help with documentation and follow-up actions even more, closing the loop from sensing to response.

### **1.3 Research gap**

Even though things have gotten better lately, there are still not many practical ways to keep an eye on fish health on a small scale. Most current methods are either too strict or too loose. At one end are comprehensive but complex systems that combine AI models with large sensor arrays [1] . These systems are very accurate, but they are also very expensive, need a lot of data, and require specialised knowledge, which makes them hard to use in outdoor ponds. On the other end, there are basic sensor networks or manual checks that keep track of water conditions but don't directly find fish or connect different factors in a way that makes it necessary to act quickly.

AI-assisted systems like camera-based mortality detectors are mostly made for controlled environments like RAS tanks [7]. They need a lot of training data and stable lighting to work well. It's hard to copy these kinds of designs in outdoor settings with limited funds. On the other hand, standard IoT monitoring (pH, temperature, etc.) gives farmers constant data streams, but it often gives them too many charts and only raises general threshold alerts. This means that it misses more subtle signs that something is wrong or the context that explains why a risk came up.

An integrated, event-aware platform that combines fish-centric observation with targeted environmental verification is what is most needed. A single fish floating or a pH drop caused by rain [8] can go unnoticed between scheduled checks because camera analytics and environmental sensing work independently. Most prototypes don't complete the alerting loop either. Even when they do find problems, they often

don't have seamless, smartphone-based push notifications that send clear, contextual information from a distance.

The specific gap this work addresses is the lack of an affordable, edge-deployable system for outdoor ponds that: detects fish mortality and unusual behaviour using vision, checks sensitive water parameters (like pH and temperature) again when an event happens (like rain), and sends real-time, contextual mobile alerts [9] to help smallholder farmers respond quickly.

This proposal aims to fill that gap by putting a Raspberry Pi-like device, an infrared camera, and a small set of sensors (pH, DS18B20 temperature, YL-83 rain, and DHT22 humidity) all under one decision engine. Camera-derived suspicions are cross-validated with water chemistry, and rain events proactively trigger focused remeasurement, shortening the detection-to-action window. Cloud messaging then sends notifications that can be acted on, along with a timestamp, readings, and an optional image as proof. The system goes beyond just parameter-only loggers and vision-only prototypes by offering a cheap, event-aware, farmer-centred solution for small-scale aquaculture.

Yes    ✓    =

No =    ✕

Partial/unclear = ~

| Work/year                             | setting                | Vision: mortality/abnormal behavior | Edge/on device inference | Cloud/mobile alerts | Cost complexity | Key limitation & new function                                                                                                                                                  |
|---------------------------------------|------------------------|-------------------------------------|--------------------------|---------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ranjgan et al./2023(Mort cam)         | RAS tanks(indoor)      | ✓                                   | ✓                        | ✓                   | High            | Turned for controlled tanks; hardware/model complexity; not designed for low-budget outdoor ponds                                                                              |
| Xiao et al.,2024(YOLO-based behavior) | Lab/production videos  | ✓                                   | ~                        | ✕                   | Med-High        | Vision focus; no farmer alert pipeline; limited discussion of outdoor deployment constraints                                                                                   |
| Li et al.,2024(DDE YOLOv9)            | Industrial aquaculture | ✓                                   | ~                        | ✕                   | Med-High        | Strong CV models but lacks alerting/user workflow; assumes more controlled conditions.                                                                                         |
| Darmalim et al.,2021(Mobile+IOT)      | Ponds                  | ✕                                   | ~                        | ✓                   | Low-Med         | It has notifications but no camera-based mortality/behavior detection.                                                                                                         |
| General IOT dashboards (various)      | Ponds/Tanks            | ✕                                   | ~                        | ~                   | Low-Med         | Parameter-centric; no fish-centric vision; alerts (if any) are generic, not image-evidenced                                                                                    |
| Proposed system (this work)           | Aquaculture Ponds      | ✓                                   | ✓<br>(Raspberry Pi)      | ✓<br>(FCM/SMS)      | Low             | Purpose-built for small outdoor ponds; camera-based mortality + abnormal behavior detection with concise, contextual phone alerts (no dependency on water quality measurement) |

Table 1.1: Comparison chart

## **1.4 Research problem**

Small outdoor fishponds still don't have an affordable, reliable way to detect dead fish and strange behaviour and send timely, relevant alerts when the weather makes things more dangerous. Current methods either concentrate on ongoing parameter logging that inundates farmers with graphs, or on camera-based detection optimised for regulated tanks, which do not adapt effectively to fluctuating outdoor environments. So, the challenge is to create and test a cost-effective, edge-deployable system that can accurately figure out when animals die and when their behaviour changes by looking at camera streams. It should also be able to link these observations to important environmental events (like rain or an unusual temperature or humidity in the enclosure) to prioritise alerts and send them to farmers in real time on their mobile devices. The solution must stay accurate even when the light changes and the surface reflects light, be able to handle sensor noise and spotty connectivity, and need little setup and maintenance so that smallholders can trust it and use it in real pond conditions.

## **2 RESEARCH OBJECTIVE**

### **2.1 Main objective**

Develop a functional prototype of an integrated monitoring and alert system for aquaculture that uses both visual fish monitoring and environmental sensing to quickly detect possible fish deaths and hazardous conditions. This will allow farm operators to take action right away.

### **2.2 Specific objective**

- Implement a Raspberry Pi-based imaging module to take periodic frames and use lightweight computer vision heuristics to figure out when fish are in an

unusual state, like when they are floating on the surface for a long time or moving very little.

- Add a sensor suite that includes a DS18B20 water-temperature probe, a pH electrode with an analog to digital interface, a YL-83 rain sensor, and a DHT22 humidity-temperature sensor to make the system aware of its surroundings.
- Design event logic that connects rainfall to immediate checks of pH and water temperature and combines evidence from different modes to cut down on false positives.
- Implement cloud connectivity to send instant push notifications through Firebase Cloud Messaging or a similar service with important readings and optional image evidence.
- Set up and carry out evaluation experiments and test cases that measure how accurate detection is, how long it takes for a reaction to happen, how reliable notifications are, and how strong the system is when it runs for a long time.

### **3 METHODOLOGY**

#### **Overview**

The project employs an iterative build-test-refine approach. A compact edge device adjacent to the pond captures video and fundamental environmental data, processes the stream locally to detect deceased fish or atypical activity, and transmits a succinct alarm to the farmer's mobile device. Every iteration enhances thresholds, [7] camera positioning, and alert messaging based on evaluations in pond-like environments.

#### **Hardware Configuration**

A Raspberry Pi Model B+ housed in a waterproof container is positioned at the pond's edge, with an infrared-enabled Raspberry Pi camera directed towards the surface area to capture floating objects. A YL-83 rain sensor is positioned in an unobstructed

location to detect precipitation, while a DHT22 sensor housed within the enclosure measures ambient temperature and humidity for contextual data. The machine operates on a rechargeable battery, optionally supplemented by solar power, with sealed wires and a securely mounted camera to reduce shaking and glare.

### **Software Workflow on the Edge Device**

The Raspberry Pi executes a lightweight application that periodically takes a frame from the camera and retrieves data from the rain and DHT22 sensors. Each frame undergoes pre-processing to stabilise brightness and conceal non-aquatic areas. The system employs basic OpenCV techniques, specifically background subtraction and frame differencing, to identify two indicators: a fish-sized area that remains relatively stationary near the surface over several frames (potential mortality), and a typically slow or solitary movement in relation to neighbouring fish (abnormal behaviour). To mitigate false alarms caused by waves, leaves, or light reflections, detections must endure for a brief duration and satisfy size and form criteria. A cool-down period inhibits the issuance of multiple warnings for the identical occurrence.

### **Contextual Framework and Decision-Making Logic**

Vision outcomes are integrated with context to enhance the utility of alerts. When rainfall occurs, detections are prioritised, and the alert message states, “detected during rainfall.” If the temperature or humidity of the enclosure is atypical, that information is provided as context. This feature does not conduct water-quality re-evaluations; it emphasises camera detection and straightforward context to ensure alarms are prompt and easily comprehensible.

## **Cloud Distribution and Mobile Notifications**

Upon event confirmation by the edge software,[9] it compiles a concise message including the event type, timestamp, a minor picture crop for verification, and relevant context tags (e.g., rainfall). This payload is transmitted to a cloud endpoint, which then delivers a notice to the farmer's mobile device. Should the network be briefly inaccessible, the device retains the warning locally and attempts reconnection automatically. Logs are stored on the device and then synced for examination.

## **Evaluation Plan**

Indoor testing starts with brief movies and arranges items to calibrate motion thresholds and persistence durations. Field testing subsequently evaluates the system's performance in both daylight and low-light conditions. The primary evaluations are: if the system accurately detected a stationary "carcass-like" target, the promptness of the alert's arrival on the phone, the frequency of false alarms triggered by waves or debris, and its ability to operate continuously for extended durations without human restarts. Thresholds and masks are calibrated until consistent performance is attained for small outdoor ponds.



### 3.1 System overview diagram



Figure 3.1: System overview

### 3.2 Component overview diagram

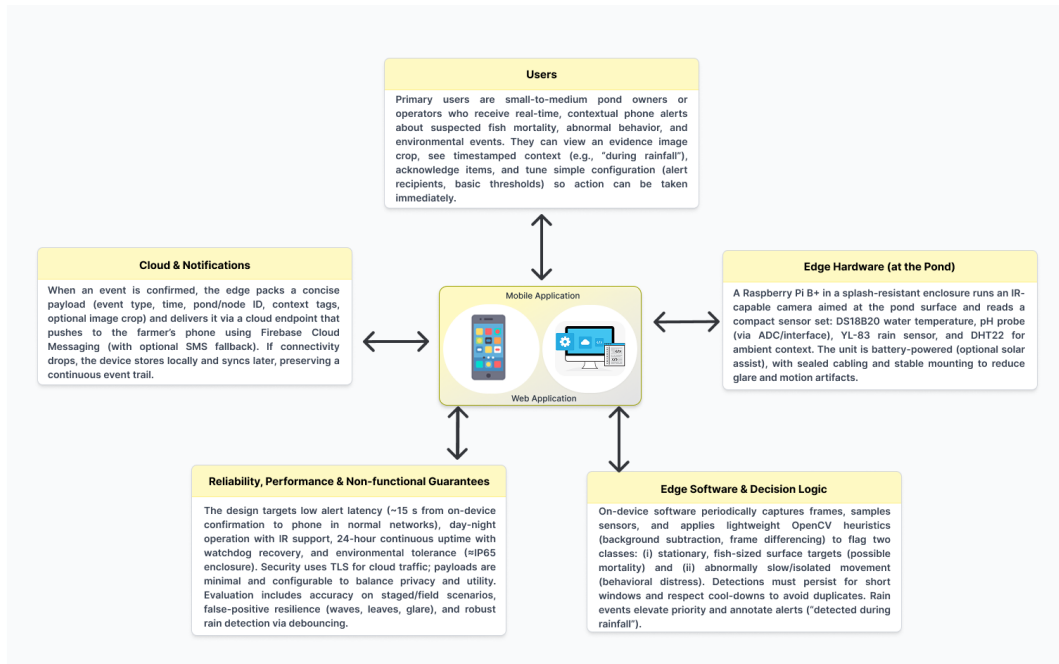


Figure 3.2: component overview

### 3.3 Gantt chart

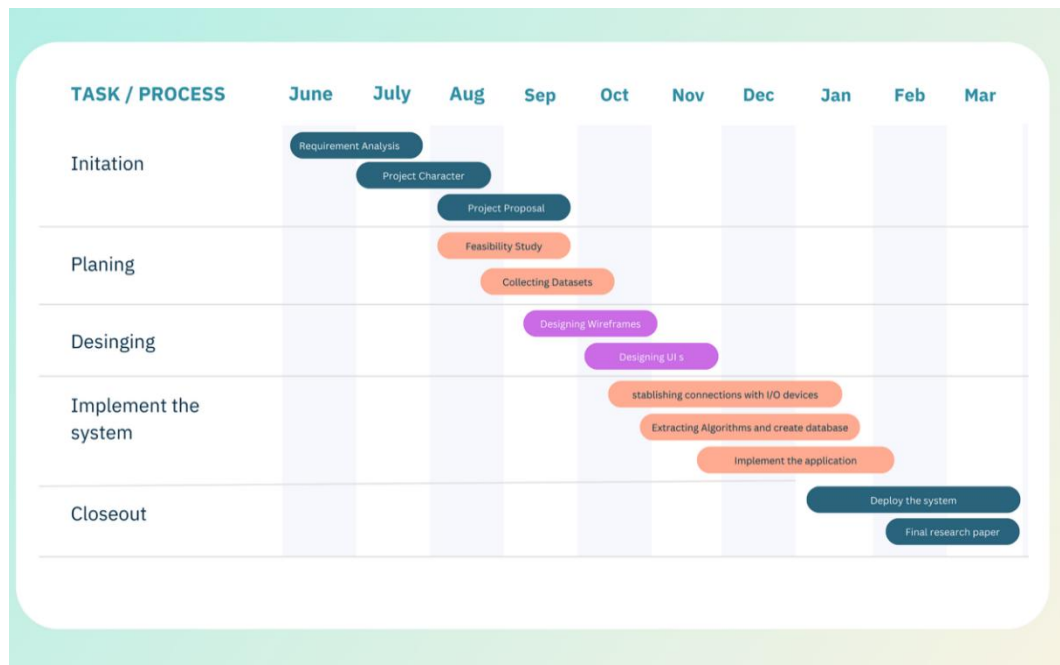


Figure 3.3: Gantt chart

## **4 PROJECT REQUIREMENTS**

### **4.1 User requirements**

- When a dead or unusual fish is suspected, view a live view and get push notifications with an image of the evidence.
- Avoid duplicate alerts by adjusting sensitivity (motion/persistence) without touching the code.
- Context should be added to alerts (e.g., "detected during rainfall").
- Day or night, the system manages waves and glare with few false alarms.
- During outages, it continues to log and queue alerts while running on edge; it synchronises later.

### **4.2 Functional requirements**

- Video Input & Monitoring: The system uses a Raspberry Pi camera that can see in IR to take pictures of the pond's surface at a low, adjustable frame rate that works day and night. This gives continuous visual coverage for detection.
- Detection of Mortality: The system finds a fish-sized area that stays almost still at or near the surface in consecutive frames and marks it as a possible death with a confidence score.
- Detection of Abnormal Behaviour: The system detects movement that is unusually low or isolated compared to other fish and raises an abnormal-behaviour event. This helps to find distress early on, before death happens.
- Persistence and Duplicate Control: Events are only confirmed after a short period of time to filter out temporary artefacts. Once an event is confirmed, duplicate alerts for the same object are not sent out for a set amount of time.

- **Rain Detection:** The YL-83 sensor is read every few seconds and debounced to tell the difference between real rain and dew or condensation. A Rain Detected alert is sent out right away when rain is detected.
- When it rains, camera sampling may go up, and vision alerts are tagged "detected during rainfall" to show how important they are and give the farmer more information.
- **Ambient Context Sensing:** A DHT22 sensor measures temperature and humidity inside an enclosure. Alerts include unusual values as context to help people understand without having to repeat the water-quality module.
- **Evidence Generation:** Verified fish events include a small image crop of the area that was detected and the confidence value so the farmer can quickly check the situation.
- **generating and Delivering Alerts:** Alerts have the type of event, the time, the node/pond ID, context tags (like rain or ambient), and evidence (for fish events). Firebase Cloud Messaging sends messages to the farmer's phone. An SMS fallback option is also available.
- **Offline Queue & Sync:** If the connection goes down, alerts and logs are stored locally and sent automatically when the connection is restored, making sure that no events are lost.

### **4.3 Non - functional requirements**

- **Performance:** All processing runs on a Raspberry Pi B+ (or something similar) without a GPU, and it keeps working well and sending alerts quickly under normal load.
- **Alert Latency:** Under normal network conditions, it should take about 15 seconds for an event to be confirmed on the device and then for a push notification to be sent to the phone.
- **Reliability of Rain Detection:** The system should send out a rain alert within a few seconds of getting wet readings that last for a while. It should also avoid false alarms when it isn't raining by using debouncing

- **Robustness and uptime:** The node should be able to run for long periods of time (at least 24 hours) without stopping, and it should be able to recover from temporary sensor or network errors and handle restarts safely.
- **Power and Autonomy:** The unit should be able to run on batteries (with optional solar assistance) in the field, and it should be able to shut down cleanly before the battery runs out, if the power module supports this.
- **Environmental Tolerance:** The enclosure and mounts should be able to handle rain and splashes (about IP65) and work in a range of normal outdoor temperatures and humidity levels.
- **Security and Privacy:** Cloud communication uses TLS, and keys and credentials are kept safe. By default, only small evidence crops, and event metadata are kept, and the length of time they are kept can be changed.
- **Usability:** Alerts are short, clear, and include proof or context so that a farmer can quickly decide what to do without having to open complicated dashboards.
- **Maintainability and interoperability:** The codebase is made up of modules for capture, vision, decision, and delivery, and it has configuration-driven thresholds and clear logs. Event payloads use a simple JSON schema, which makes it easy for other modules, like water quality, to work with them.
- **Cost Constraint:** The hardware needed for this function (Pi, IR camera, YL-83, DHT22, enclosure, and power) should stay within a low budget that is easy for students to work with.

#### 4.4 Test cases

| Test Case ID | Task                                | Test Objective                                                                       | Expected Result                                                                                             | Status |
|--------------|-------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------|
| TC_01        | Even feed distribution              | To verify the uniform distribution of feed across the pond using trays.              | Equal feed amounts collected across trays; no dominant zones. $CV \leq 25\%$ .                              | Passed |
| TC_02        | Accuracy of Dosing (Closed-Loop)    | To test if the system dispenses the exact requested mass.                            | actual dispensed mass is within $\pm 10\%$ of target. Top-up is triggered if underdosed.                    | Passed |
| TC_03        | Low-Battery Interlock               | To ensure the system blocks feeding commands when the battery is critically low.     | Feeding is blocked within 2s. Alert "Low battery feed inhibited" appears. Event is logged with voltage.     | Passed |
| TC_04        | Detect Jam or Empty Hopper          | To detect obstructions or when the hopper is empty.                                  | Motor stops. Alert shown: "Motor jam" or "Hopper empty." Reason logged.                                     | Passed |
| TC_05        | Schedule + Remote Control with Sync | To validate scheduled feed during offline and successful sync & control once online. | Offline feed runs, logs are saved and synced later. Remote command works after reconnect with <15s latency. | Passed |

Table 4.1: Test case

## 5 BUDGET AND JUSTIFICATION

| Device                                | Price (Rs)      | Usability                                         |
|---------------------------------------|-----------------|---------------------------------------------------|
| Raspberry Pi Camera Module            | 3500.00         | Used for live footage and fish movement detection |
| DS18B20 Waterproof Temperature Sensor | 230.00          | Temperature monitoring                            |
| YL-83 Rain Sensor Module              | 950.00          | Rain detection for environmental context          |
| DHT22 Humidity Sensor                 | 600.00          | Humidity monitoring for environmental changes     |
| Raspberry Pi B+                       | 23500.00        | Microcontroller for processing                    |
| <b>Component Total Budget</b>         | <b>28780.00</b> |                                                   |

Table 5.1: Budget and justification

This bill of materials is the least number of things needed to run vision-based mortality/behavior detection with event-driven environmental context. The Raspberry Pi B+ has enough power to run OpenCV motion and float detection and send alerts. The Raspberry Pi Camera Module, on the other hand, provides continuous images that are needed to flag a dead or unusually slow or isolated fish. The DS18B20 waterproof temperature sensor adds direct in-water temperature context (a primary stressor) to enrich alerts without the cost/complexity of a full water-chemistry stack in this module. The YL-83 rain sensor picks up on the start of rain so that the system can tighten checks and re-evaluate conditions right after dilution events. The DHT22 humidity/air-temperature sensor, on the other hand, picks up on changes in the greenhouse or outside air that often happen before changes in surface behavior. These parts work together to provide edge CV detection and lightweight, event-aware environmental monitoring that fits within the scope of this function. They cost Rs 28,780 and keep the design low-cost, durable, and easy to set up.

## **6 COMMERCIALIZATION AND MARKET FEASIBILITY**

- Customers include pond farmers who are unable to keep a close eye on their outdoor or greenhouse ponds.
- Value: documented evidence supports decision-making; early action minimizes unreported deaths and labor for manual checks.
- Differentiators include edge processing (low bandwidth), mobile-first alerts, and camera-first detection with context tags (rain/temperature).
- Indicative cost and price: core kit ~Rs 28.8k; includes device and app access; optional maintenance plan.
- Risks and ways to mitigate them include connectivity → store-and-forward; false positives → persistence/hysteresis; lighting changes → IR cam + masks.

## 7 REFERENCES

- [1] R. Ranjan, K. Sharrer, S. Tsukuda, and C. Good, “MortCam: An Artificial Intelligence-aided fish mortality detection and alert system for recirculating aquaculture,” *Aquacultural Engineering*, vol. 102, p. 102341, 2023, doi: 10.1016/j.aquaeng.2023.102341.
- [2] R. P. Shete, S. A. Shelar, and S. V. Lokhande, “IoT-enabled effective real-time water quality monitoring in aquaculture,” *J. King Saud Univ. – Comput. Inf. Sci.*, 2024. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11387385/>. Accessed: Aug. 29, 2025.
- [3] N. A. M. Mohd Jais et al., “Improved accuracy in IoT-based water quality monitoring for aquaculture tanks using low-cost sensors: Asian seabass fish farming,” *Heliyon*, vol. 10, no. 8, e29022, 2024, doi: 10.1016/j.heliyon.2024.e29022.
- [4] M. M. Hemal et al., “An Integrated Smart Pond Water Quality Monitoring and Recommendation System (AquaBot),” *Sensors*, vol. 24, no. 11, 3682, 2024, doi: 10.3390/s24113682.
- [5] K. Xiao, R. Li, and S. Lin, “Fish jumping behavior automatic recognition using YOLOv5,” *Aquatic Toxicology*, 2024, Article 107137. [Online]. Available: PubMed 39520842. Accessed: Aug. 29, 2025.
- [6] Y. Li et al., “DDEYOLOv9: Network for detecting and counting abnormal fish behavior in complex water environments,” *Fishes*, vol. 9, no. 6, 242, 2024, doi: 10.3390/fishes9060242.
- [7] L. H. Sipaúba-Tavares and F. M. S. Braga, “Effects of rainfall on water quality in sequentially disposed fishponds,” *Brazilian J. Biology*, vol. 67, no. 4, pp. 643–649, 2007, doi: 10.1590/S1519-69842007000400008.
- [8] A. J. Makori, I. Abuom, R. Kapiyo, D. Anyona, and P. Dida, “Effects of water physico-chemical parameters on tilapia performance,” *Fisheries and Aquatic Sciences*, vol. 20, 47, 2017, doi: 10.1186/s41240-017-0075-7.
- [9] F. Darmalim, A. A. Hidayat, T. W. Cenggoro, K. Purwandi, S. Darmalim, and B. Pardamean, “An integrated system of mobile application and IoT solution for pond monitoring,” *IOP Conf. Ser.: Earth Environ. Sci.*, vol. 794, 012106, 2021, doi: 10.1088/1755-1315/794/1/012106.
- [10] M. Miller, A. Kisiel, D. Cembrowska-Lech, I. Durlik, and T. Miller, “IoT in water quality monitoring” *Sensors*, vol. 23, no. 2, 960, 2023, doi: 10.3390/s23020960.



## 8 APPENDIX

